Diffusion Policy

VLab
DGIST Computer Vision Laboratory

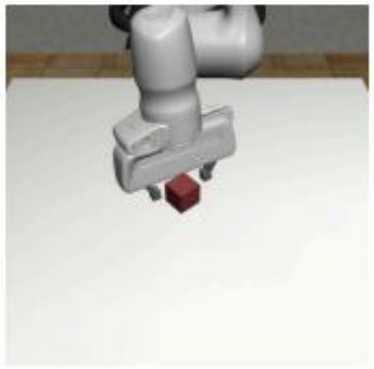
Eunsong Kim

Diffusion Policy:
Visuomotor Policy Learning via Action Diffusion

Cheng Chi[1], Siyuan Feng[2], Yilun Du[3], Zhenjia Xu[1], Eric Cousineau[2], Benjamin Burchfiel[2], Shuran Song[1]
[1] Columbia University [2] Toyota Research Institute [3] MIT

Diffusion Policy

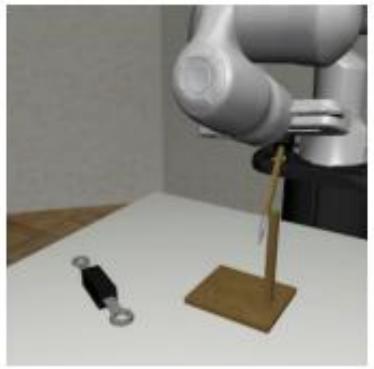
- **Task**


Lift

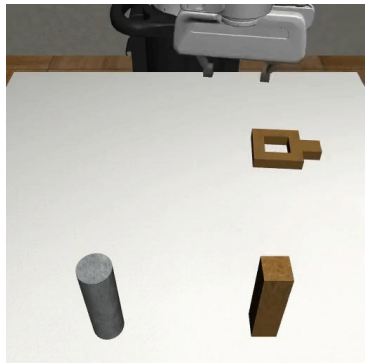

Can

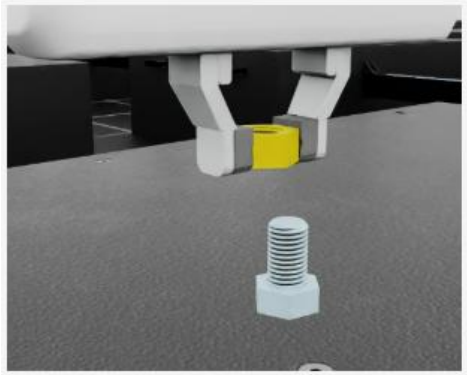

Square


Transport


Tool Hang


Push-T



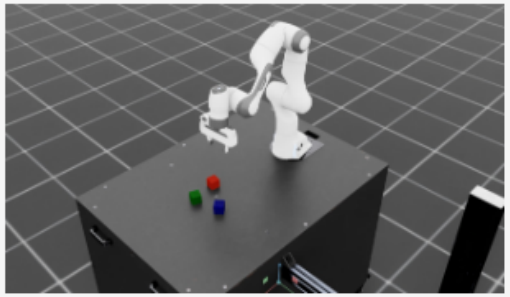


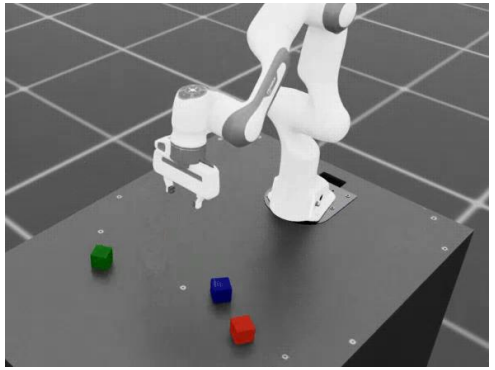
Isaac-Factory-NutThread-Direct-v0

Thread the nut onto the first 2 threads of the bolt, using the Franka robot

Diffusion Policy

- **Task**
  - Stack cube, Franka
  - 키보드, demo 10개

	Isaac-Stack-Cube-Franka-v0 Isaac-Stack-Cube-Franka-IK-Rel-Blueprint-v0	Stack three cubes (bottom to top: blue, red, green) with the Franka robot. Blueprint env used for the NVIDIA Isaac GR00T blueprint for synthetic manipulation motion generation



Diffusion Policy

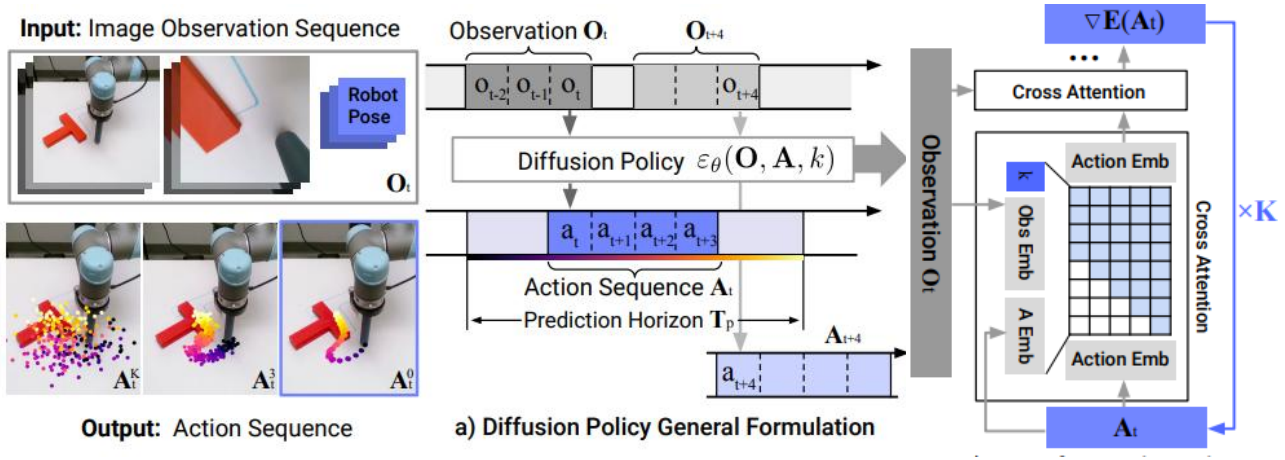
- **Lowdim**
  - Input: cube orientations (T, 12), cube positions (T, 9), eef pos (T, 3), eef quat (T, 4), gripper pos (T, 2), joint pos (T, 9), jo int vel (T, 9) => (T, 48)
  - Output: actions (T, 7)

Diffusion Policy

- **Lowdim**
  - 카메라 사용 x, encoder x

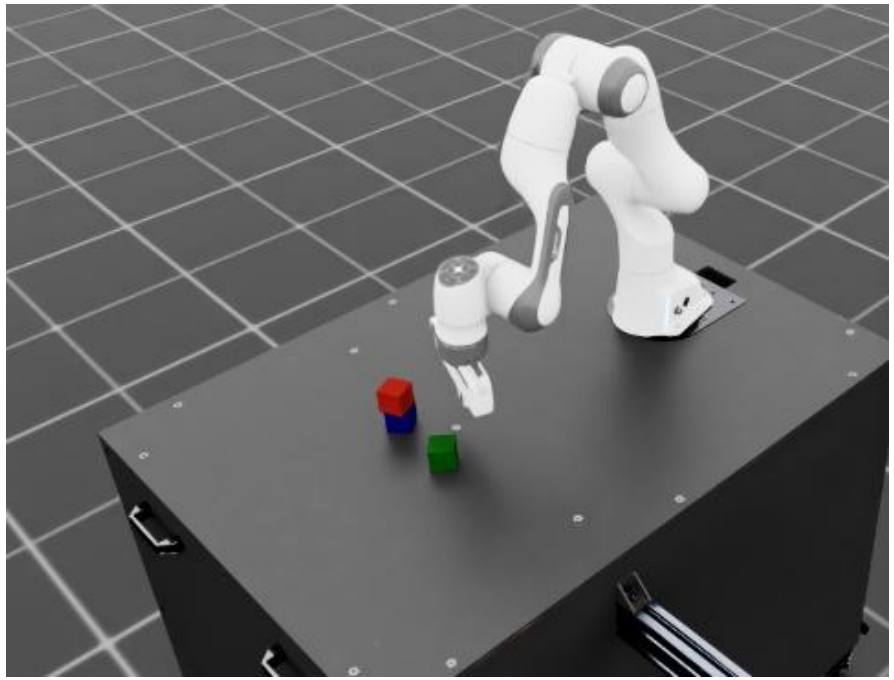
```python
# handle different ways of passing observation
cond = None
trajectory = action
if self.obs_as_cond:
    cond = obs[:,:self.n_obs_steps,:]
    if self.pred_action_steps_only:
        To = self.n_obs_steps
        start = To - 1
        end = start + self.n_action_steps
        trajectory = action[:,start:end]
else:
    trajectory = torch.cat([action, obs], dim=-1)
```

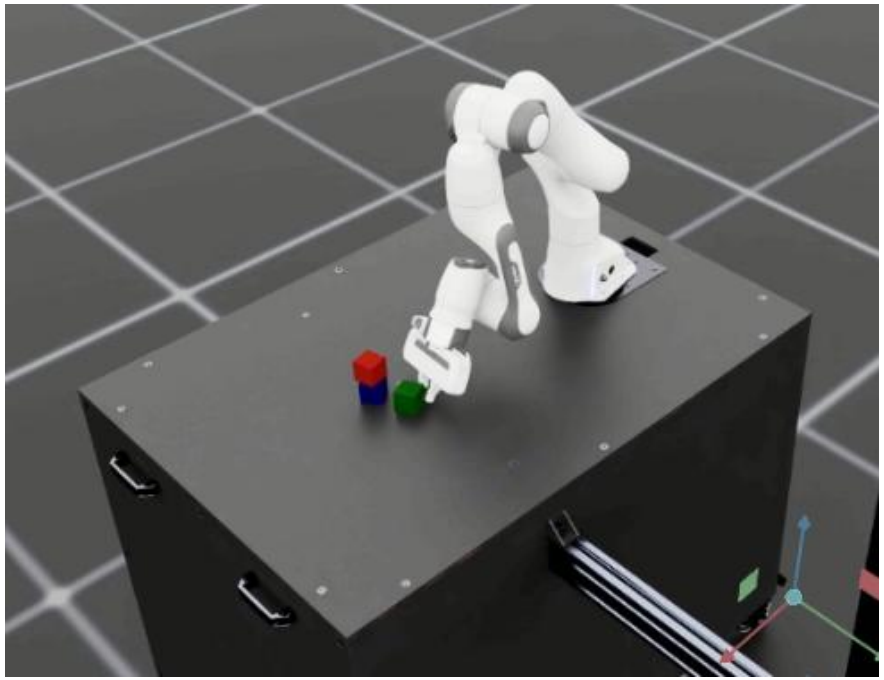
- Noise scheduler - DDPM
- Noise estimator - transformer



Diffusion Policy

- **Lowdim Result**

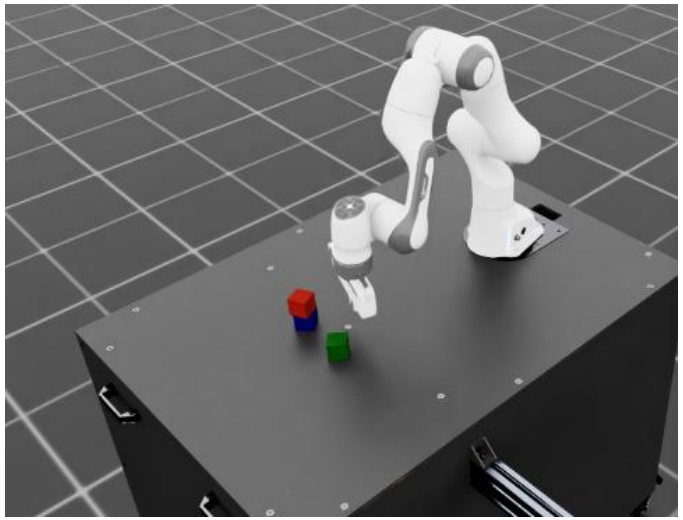
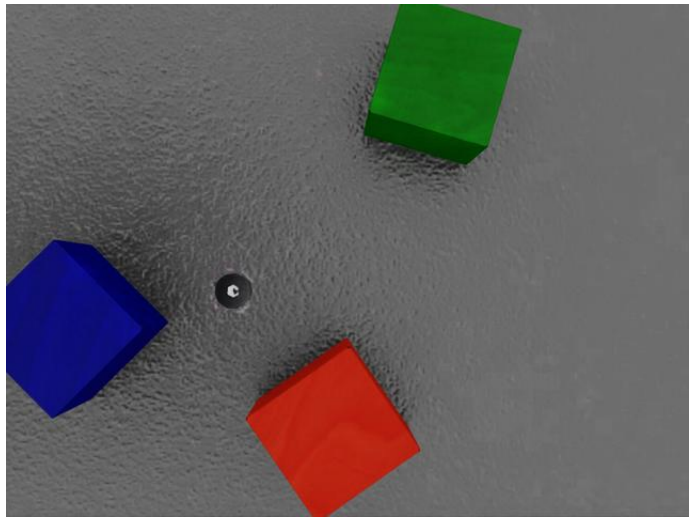
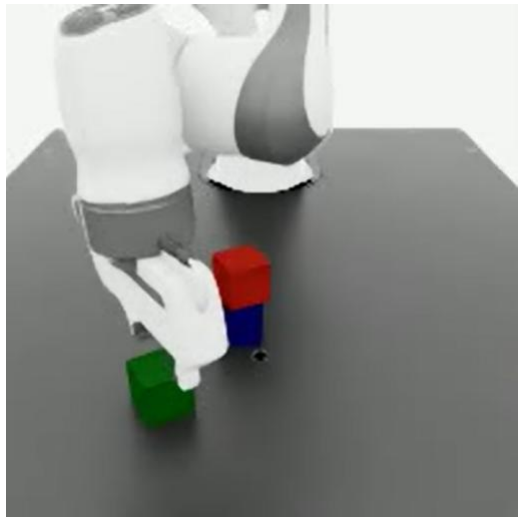




Diffusion Policy

- **RGB**
  - Input: eef pos (T, 3), eef quat (T, 4), gripper pos (T, 2) => (T, 9) / table cam (T, 3, 84, 84), wrist cam (T, 3, 84, 84)
  - Output: actions (T, 7)

  - 전체 table 구성, wrist cam RGB, table cam RGB

Diffusion Policy

- **RGB**
  - 카메라 사용, encoder
  - MultiImageObsEncoder

```
202         # handle different ways of passing observation
203         local_cond = None
204         global_cond = None
205         trajectory = nactions
206         cond_data = trajectory
207         if self.obs_as_global_cond:
208             # reshape B, T, ... to B*T
209             this_nobs = dict_apply(nobs,
210                 lambda x: x[:,:self.n_obs_steps,...].reshape(-1,*x.shape[2:]))
211             nobs_features = self.obs_encoder(this_nobs)
212             # reshape back to B, Do
213             global_cond = nobs_features.reshape(batch_size, -1)
214         else:
215             # reshape B, T, ... to B*T
216             this_nobs = dict_apply(nobs, lambda x: x.reshape(-1, *x.shape[2:]))
217             nobs_features = self.obs_encoder(this_nobs)
218             # reshape back to B, T, Do
219             nobs_features = nobs_features.reshape(batch_size, horizon, -1)
220             cond_data = torch.cat([nactions, nobs_features], dim=-1)
221             trajectory = cond_data.detach()
222
```

```
class MultiImageObsEncoder(ModuleAttrMixin):
    def __init__(self,
                 shape_meta: dict,
                 rgb_model: Union[nn.Module, Dict[str,nn.Module]],
                 resize_shape: Union[Tuple[int,int], Dict[str,tuple], None]=None,
                 crop_shape: Union[Tuple[int,int], Dict[str,tuple], None]=None,
                 random_crop: bool=True,
                 # replace BatchNorm with GroupNorm
                 use_group_norm: bool=False,
                 # use single rgb model for all rgb inputs
                 share_rgb_model: bool=False,
                 # renormalize rgb input with imagenet normalization
                 # assuming input in [0,1]
                 imagenet_norm: bool=False
        ):
        """
        Assumes rgb input: B,C,H,W
        Assumes low_dim input: B,D
        """
```

Diffusion Policy

- **RGB**
  - Resize, crop, normalize
  - Resnet18

```python
def forward(self, obs_dict):
    batch_size = None
    features = list()
    # process rgb input
    if self.share_rgb_model:
        # pass all rgb obs to rgb model
        imgs = list()
        for key in self.rgb_keys:
            img = obs_dict[key]
            if batch_size is None:
                batch_size = img.shape[0]
            else:
                assert batch_size == img.shape[0]
            assert img.shape[1:] == self.key_shape_map[key]
            img = self.key_transform_map[key](img)
            imgs.append(img)
        # (N*B,C,H,W)
        imgs = torch.cat(imgs, dim=0)
        # (N*B,D)
        feature = self.key_model_map['rgb'](imgs)
        # (N,B,D)
        feature = feature.reshape(-1,batch_size,*feature.shape[1:])
        # (B,N,D)
        feature = torch.moveaxis(feature,0,1)
        # (B,N*D)
        feature = feature.reshape(batch_size,-1)
        features.append(feature)
    else: # rgb obs에 대해 각각 다른 model을 사용함
        # run each rgb obs to independent models
        for key in self.rgb_keys: # table_cam, wrist_cam
            img = obs_dict[key]
            if batch_size is None:
                batch_size = img.shape[0]
            else:
                assert batch_size == img.shape[0]
            assert img.shape[1:] == self.key_shape_map[key]
            img = self.key_transform_map[key](img)
            feature = self.key_model_map[key](img)
            features.append(feature)
```

```python
    # process lowdim input
    for key in self.low_dim_keys:
        data = obs_dict[key]
        if batch_size is None:
            batch_size = data.shape[0]
        else:
            assert batch_size == data.shape[0]
        assert data.shape[1:] == self.key_shape_map[key]
        features.append(data)

    # concatenate all features
    result = torch.cat(features, dim=-1) # rgb와 low_dim 데이터를 모두 합쳐줌
    return result
```

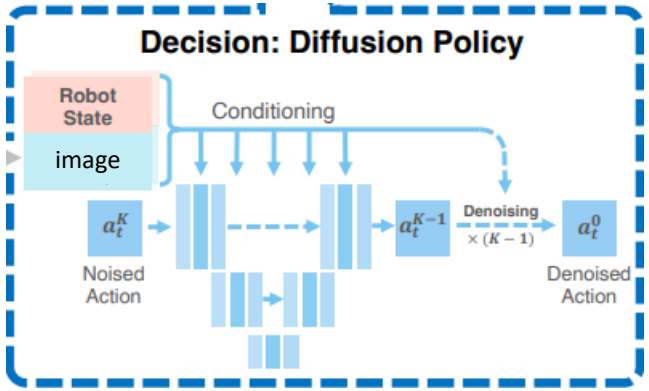
```yaml
obs_encoder:
  _target_: diffusion_policy.model.vision.multi_image_obs_encoder.MultiImageObsEncoder
  shape_meta: ${shape_meta}
  rgb_model:
    _target_: diffusion_policy.model.vision.model_getter.get_resnet
    name: resnet18
    weights: null
  resize_shape: null
  crop_shape: [76, 76]
  # constant center crop
  random_crop: True
  use_group_norm: True
  share_rgb_model: False
  # imagenet_norm: True
  imagenet_norm: False
```

Diffusion Policy

- **RGB**
  - Noise scheduler – DDPM
  - Noise estimator – ConditionalUnet1D

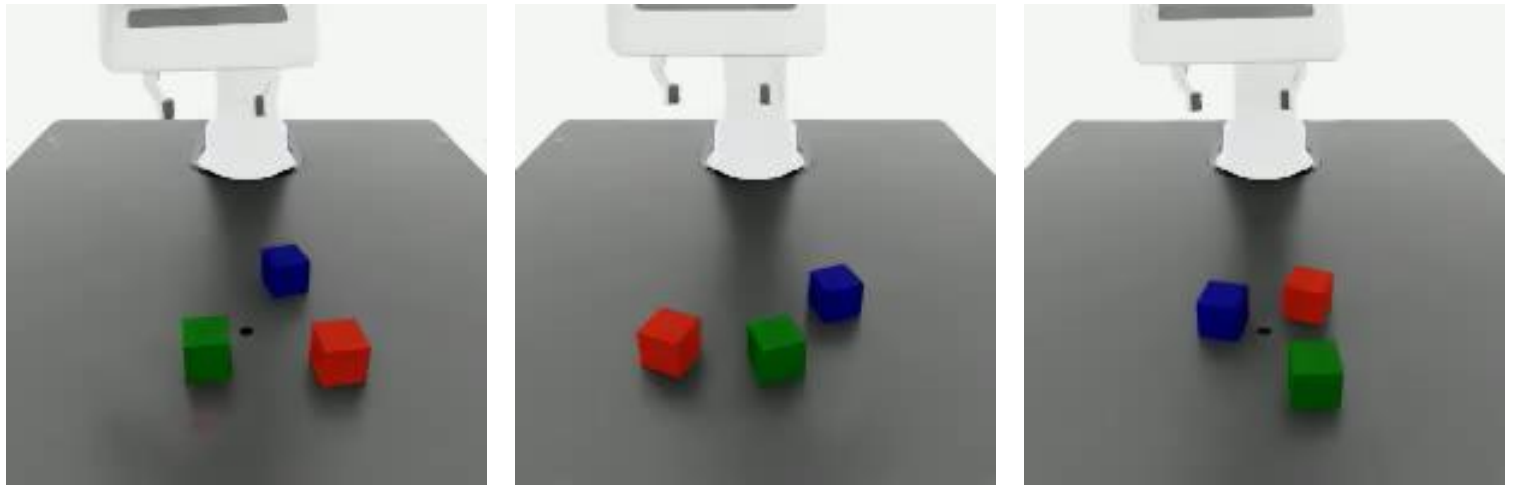
```python
class DiffusionUnetImagePolicy(BaseImagePolicy):
    def __init__(self,
            shape_meta: dict,
            noise_scheduler: DDPMScheduler,
            obs_encoder: MultiImageObsEncoder,
            horizon,
            n_action_steps,
            n_obs_steps,
            num_inference_steps=None,
            obs_as_global_cond=True, # action만 Unet의
            # 이게 False일 경우 action과 obs가 Unet의 입력

            diffusion_step_embed_dim=256,
            down_dims=(256,512,1024),
            kernel_size=5,
            n_groups=8,
            cond_predict_scale=True,
            # parameters passed to step
            **kwargs):
```

```python
model = ConditionalUnet1D(
    input_dim=input_dim,
    local_cond_dim=None,
    global_cond_dim=global_cond_dim,
    diffusion_step_embed_dim=diffusion_step_embed_dim,
    down_dims=down_dims,
    kernel_size=kernel_size,
    n_groups=n_groups,
    cond_predict_scale=cond_predict_scale
)
```



Diffusion Policy

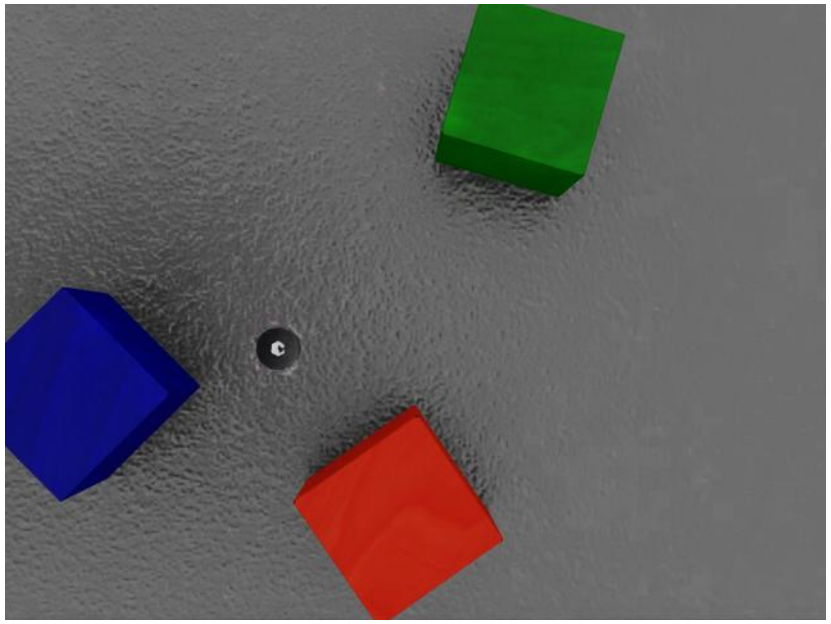
- **RGB Result**

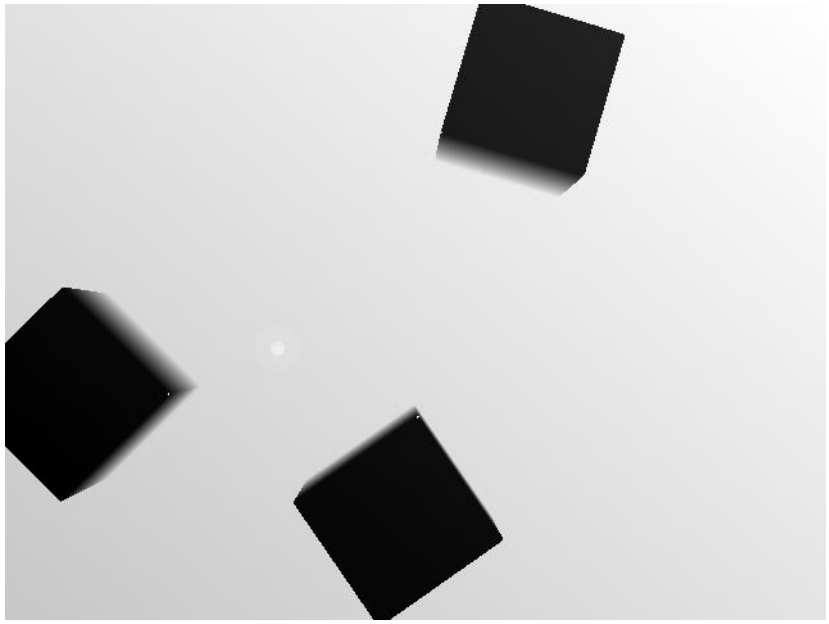


Diffusion Policy

- **RGBD**
  - Input: eef pos (T, 3), eef quat (T, 4), gripper pos (T, 2) => (T, 9) / table cam (T, 4, 84, 84), wrist cam (T, 4, 84, 84)
  - Output: actions (T, 7)

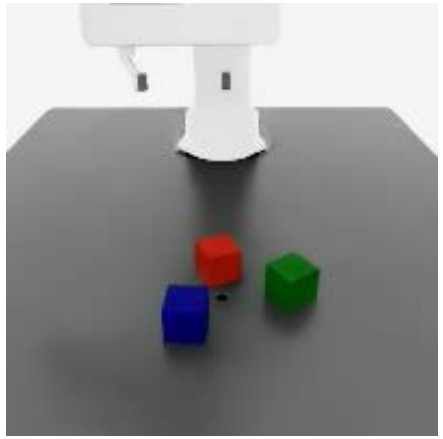
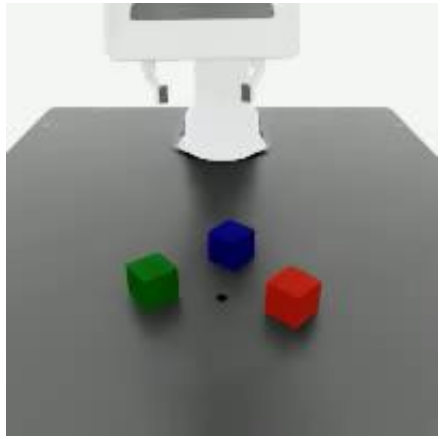
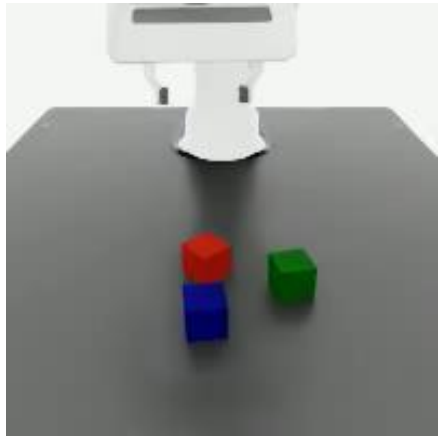
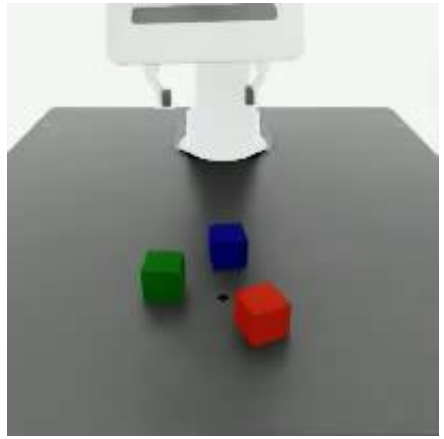
  - RGB + D

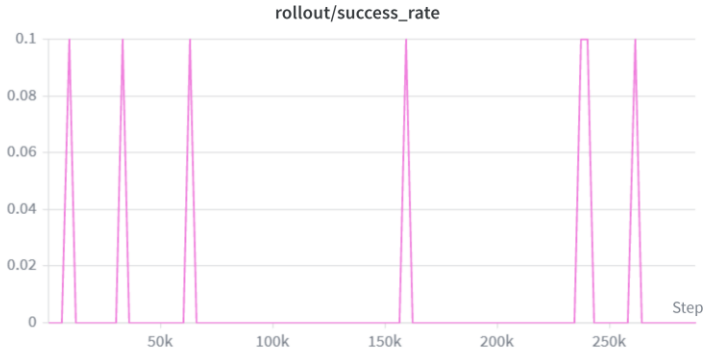




Diffusion Policy

- **RGBD Result**



3D Diffusion Policy:
Generalizable Visuomotor Policy Learning via Simple 3D Representations

Yanjie Ze[1*] Gu Zhang[12*] Kangning Zhang[12] Chenyuan Hu[13] Muhan Wang[13] Huazhe Xu[314]
[1]Shanghai Qi Zhi Institute [2]Shanghai Jiao Tong University [3]Tsinghua University, IIIS [4]Shanghai AI Lab
[*]Equal contribution

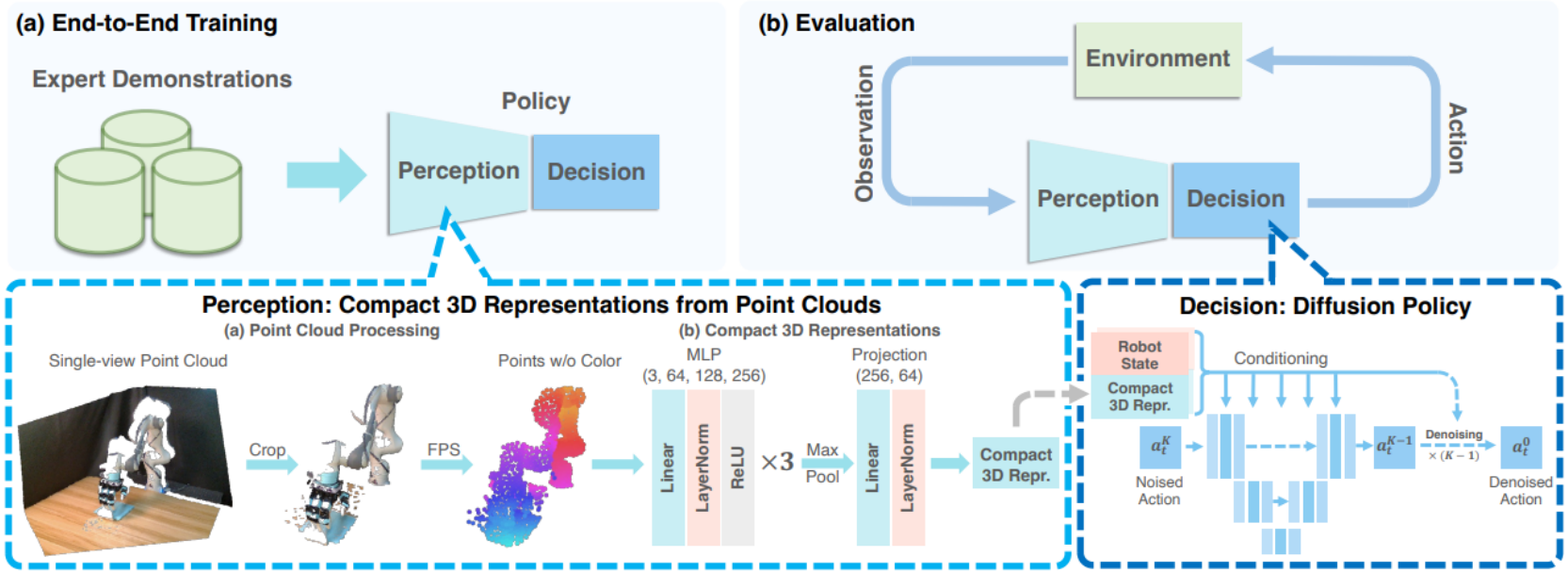
3D Diffusion Policy

- **Point cloud, no color**
  - Input: eef pos (T, 3), eef quat (T, 4), gripper pos (T, 2) => (T, 9) / point cloud (T, 2048, 6)
  - Output: actions (T, 7)

```python
def preprocess_pointcloud(pointcloud, num_points=512):
    xmin, xmax = -2.0, 2.0
    ymin, ymax = -2.0, 2.0
    zmin, zmax = 0.01, 2.0

    processed = []

    for pc in pointcloud:
        mask = (
            (pc[:,0] > xmin) & (pc[:,0] < xmax) &
            (pc[:,1] > ymin) & (pc[:,1] < ymax) &
            (pc[:,2] > zmin) & (pc[:,2] < zmax)
        )
        pc = pc[mask]

        if pc.shape[0] >= num_points:
            sampled = []
            sampled_idx = np.zeros(num_points, dtype=int)
            sampled_idx[0] = np.random.randint(pc.shape[0])
            coords = pc[:, :3]
            distances = np.linalg.norm(coords - coords[sampled_idx[0]], axis=1)

            for i in range(1, num_points):
                sampled_idx[i] = np.argmax(distances)
                dist_new = np.linalg.norm(coords - coords[sampled_idx[i]], axis=1)
                distances = np.minimum(distances, dist_new)
            pc_sampled = pc[sampled_idx]

        else:
            pad = num_points - pc.shape[0]
            pc_sampled = np.pad(pc, ((0,pad),(0,0)))

        processed.append(pc_sampled)

    return np.stack(processed)

table_pointcloud = preprocess_pointcloud(g["obs"]["table_cam_pointcloud"][:], 2048)
pointcloud_list.append(table_pointcloud)

pos = np.concatenate([g["obs"]["eef_pos"][:], g["obs"]["eef_quat"][:], g["obs"]["gripper_pos"][:]], axis=-1)
agent_pos_list.append(pos)

action_list.append(g["actions"][:])
total_steps += g["actions"].shape[0]
episode_ends.append(total_steps)
```

3D Diffusion Policy

- **Point cloud, no color**



3D Diffusion Policy

- **Point cloud, no color**
  - encoder

```python
class PointNetEncoderXYZ(nn.Module):
    def __init__(self,

        self.mlp = nn.Sequential(
            nn.Linear(in_channels, block_channel[0]),
            nn.LayerNorm(block_channel[0]) if use_layernorm else nn.Identity(),
            nn.ReLU(),
            nn.Linear(block_channel[0], block_channel[1]),
            nn.LayerNorm(block_channel[1]) if use_layernorm else nn.Identity(),
            nn.ReLU(),
            nn.Linear(block_channel[1], block_channel[2]),
            nn.LayerNorm(block_channel[2]) if use_layernorm else nn.Identity(),
            nn.ReLU(),
        )


        if final_norm == 'layernorm':
            self.final_projection = nn.Sequential(
                nn.Linear(block_channel[-1], out_channels),
                nn.LayerNorm(out_channels)
            )
        elif final_norm == 'none':
            self.final_projection = nn.Linear(block_channel[-1], out_channels)
        else:
            raise NotImplementedError(f"final_norm: {final_norm}")

        self.use_projection = use_projection
        if not use_projection:
            self.final_projection = nn.Identity()
            cprint("[PointNetEncoderXYZ] not use projection", "yellow")

        VIS_WITH_GRAD_CAM = False
        if VIS_WITH_GRAD_CAM:
            self.gradient = None
            self.feature = None
            self.input_pointcloud = None
            self.mlp[0].register_forward_hook(self.save_input)
            self.mlp[6].register_forward_hook(self.save_feature)
            self.mlp[6].register_backward_hook(self.save_gradient)
```

```python
    def forward(self, x):
        x = self.mlp(x)
        x = torch.max(x, 1)[0]
        x = self.final_projection(x)
        return x
```

3D Diffusion Policy

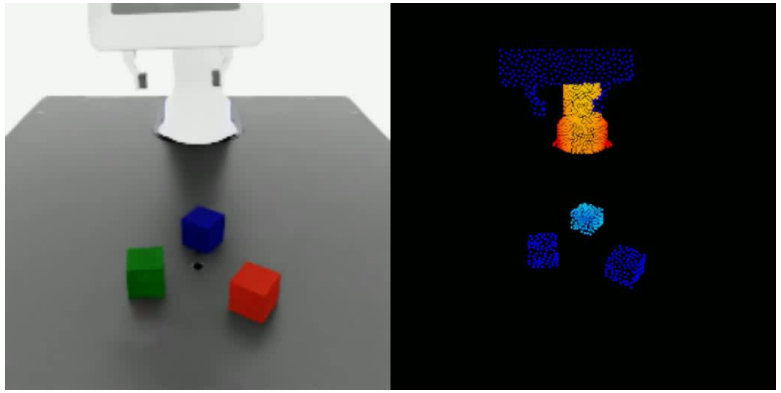
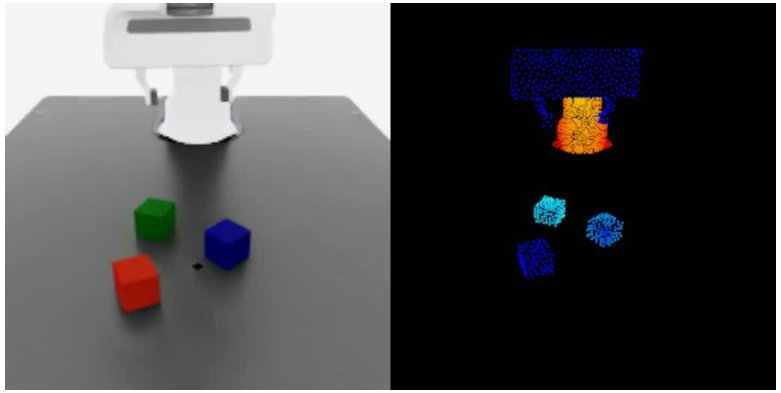
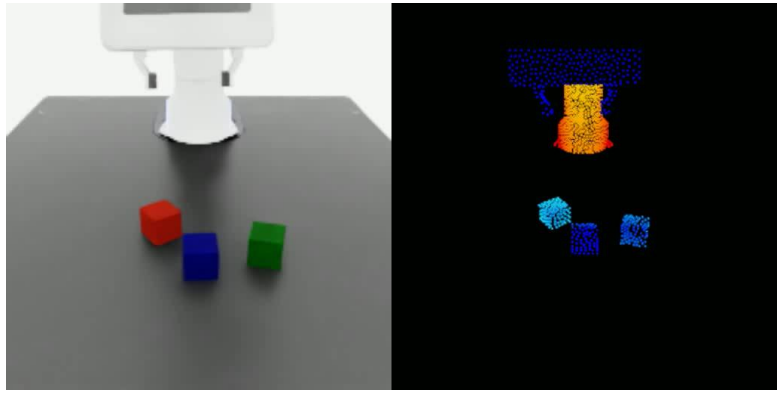
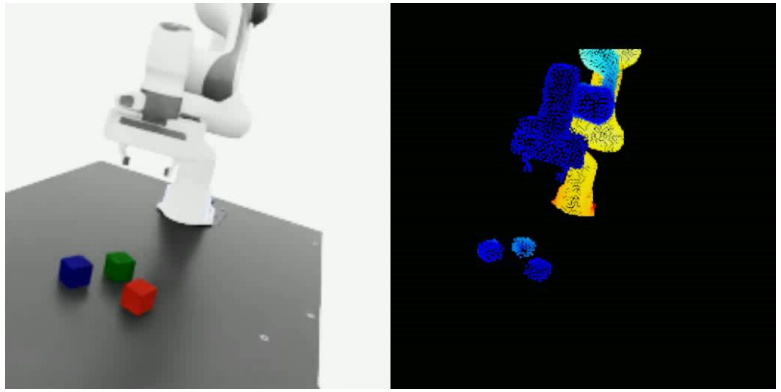
- **Point cloud, no color**
  - Noise scheduler - DDPM
  - Noise estimator – ConditionalUnet1D

```python
class DP3(BasePolicy):
    def __init__(self,
            shape_meta: dict,
            noise_scheduler: DDPMScheduler,
            horizon,
            n_action_steps,
            n_obs_steps,
            num_inference_steps=None,
            obs_as_global_cond=True,
            diffusion_step_embed_dim=256,
            down_dims=(256,512,1024),
            kernel_size=5,
            n_groups=8,
            condition_type="film",
            use_down_condition=True,
            use_mid_condition=True,
            use_up_condition=True,
            encoder_output_dim=256,
            crop_shape=None,
            use_pc_color=False,
            pointnet_type="pointnet",
            pointcloud_encoder_cfg=None,
            # parameters passed to step
            **kwargs):
```

```python
model = ConditionalUnet1D(
    input_dim=input_dim,
    local_cond_dim=None,
    global_cond_dim=global_cond_dim,
    diffusion_step_embed_dim=diffusion_step_embed_dim,
    down_dims=down_dims,
    kernel_size=kernel_size,
    n_groups=n_groups,
    condition_type=condition_type,
    use_down_condition=use_down_condition,
    use_mid_condition=use_mid_condition,
    use_up_condition=use_up_condition,
)
```

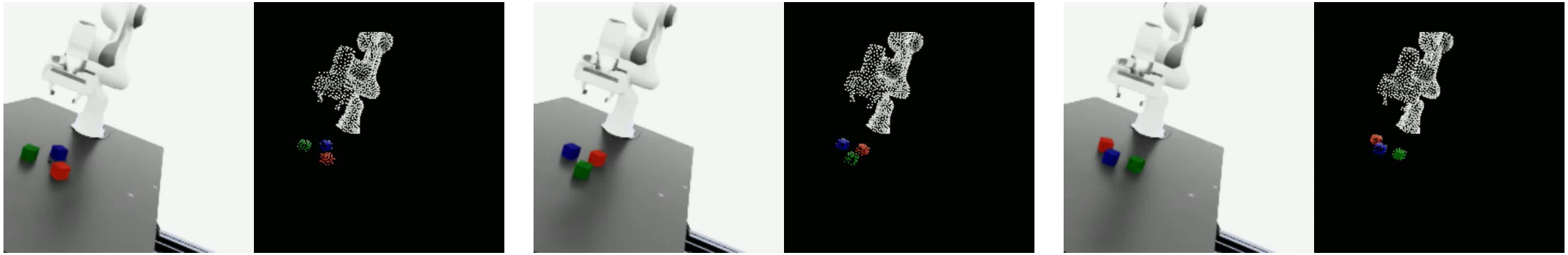
3D Diffusion Policy

- **Point cloud, no color Result**

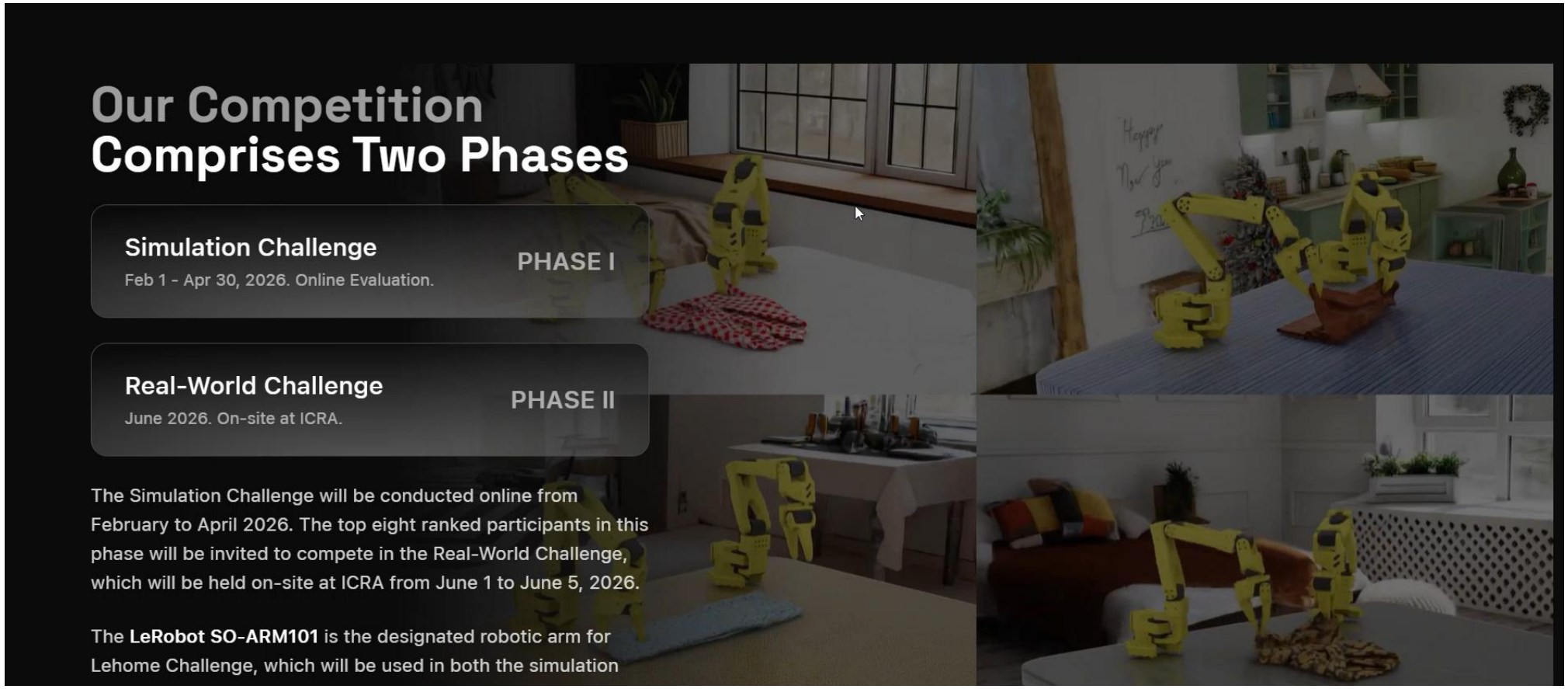
3D Diffusion Policy

- **Point cloud, color Result**



Diffusion Policy

- **LeHome challenge**



Thank you

DGIST 대구경북과학기술원
VLab DGIST 컴퓨터 비전 연구실